МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"
Кафедра «Систем автоматизованого проектування»



Звіт

до лабораторної роботи №8

з дисципліни "Технології веб розробки та дизайну"

Проєкт «MYSTETSVO»

на тему:

Деплой проєкту. Контроль версій Git та GitHub

Віддалений репозиторій на GitHub:	`https://github.com/dbyrba/mystetstvo`
Опублікований сайт на GitHub Pages:	`https://dbyrba.github.io/mystetstvo/`
Опублікований сайт на хостингу Netlify:	`https://mystetstvo-hub.netlify.app`

Виконала:
ст. гр. ПП-25
Бурба Д.Р.

Перевірила:
к.т.н., доц. Паращин Ж.Д.

Львів – 2026

Мета роботи:

Опанувати фундаментальні інженерні навички роботи з розподіленою системою контролю версій Git та провідним хмарним сервісом GitHub через термінал і графічний інтерфейс IDE Visual Studio Code. Навчитися ініціалізувати локальні репозиторії, керувати станами файлів у Staging Area, формувати атомарні фіксації (commit) за конвенціями версіонування, реалізовувати стратегії розгалуження (branching), здійснювати безконфліктне злиття (merge), організовувати спільну командну розробку та виконувати багаторівневий деплой (deployment) статичного багатосторінкового вебресурсу на платформи GitHub Pages та Netlify Edge CDN.

Стек інструментів для роботи:

- **Visual Studio Code (v1.94+):** Професійне середовище веб-розробки з інтегрованим терміналом PowerShell та візуальною панеллю Source Control.
- **Git CLI (v2.47.0.windows.2):** Локальна розподілена система керування версіями сирцевого коду з повною підтримкою розгалуження, фіксацій та аналізу історії.
- **GitHub (Акаунт dbyrba):** Хмарна платформа для віддаленого зберігання Git-репозиторіїв, управління доступом, релізування та організації колаб-розробки.
- **GitHub Pages:** Сервіс автоматизованого хостингу статичних сайтів безпосередньо з гілки main віддаленого репозиторію.
- **Netlify Edge CDN:** Хмарна платформа безперервної доставки (Continuous Deployment) з глобальною дистрибуцією, автоматичним SSL (Let's Encrypt) та кешуванням.
- **Генеративні ШІ-моделі (Gemini 2.5 Pro / Claude 3.5 Sonnet / Copilot):** Інтелектуальні асистенти для складання інженерних промптів, автоматизації версіонування та оптимізації деплою.

Характеристики бізнес-проєкту «MYSTETSVO»:

- Тематика: Цифровий хаб українського візуального мистецтва (оцифрування національної спадщини, відкритий провенанс творів, інтерактивна афіша подій та реєстр резидентів);
- Архітектурна структура (5 сторінок): Головна (index.html) з відеоманіфестом, Галерея робіт (gallery.html) з табами та відеоартом, Календар подій (events.html) з RSVP, Реєстр митців (artists.html) з портретами та Про хаб (about.html) з аналітикою та контактами;
- Мультимедійне наповнення: Інтегрований авторський генеративний відеоарт «Трансформація форми: Код спадщини» (HD 720p 60fps MP4, 3.68 МБ) з кастомним UI-плеєром;
- Авторський акаунт на GitHub: dbyrba, назва віддаленого репозиторію: mystetstvo, публічний статус доступу.

РОЗДІЛ 1. ВСТАНОВЛЕННЯ ТА КОНФІГУРАЦІЯ GIT І АКАУНТА GITHUB

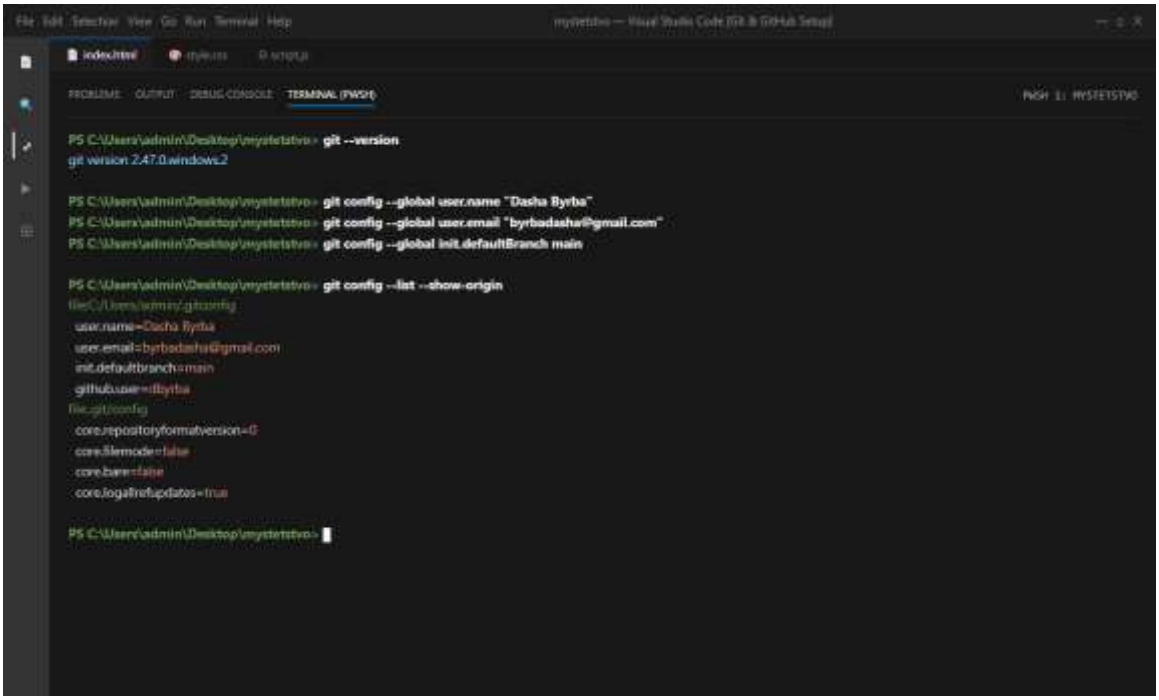
1.1. Інженерне розмежування понять Git та GitHub

У процесі виконання лабораторної роботи було чітко розмежовано фундаментальні поняття вебінженерії:
1) Git — це локальна розподілена система керування версіями (VCS), що функціонує безпосередньо на робочій станції розробника, фіксує знімки (snapshots) стану файлової системи, веде історію змін та дозволяє ізольовано працювати над функціоналом через механізм гілок;
2) GitHub — це глобальна хмарна екосистема для хостингу віддалених репозиторіїв Git, що забезпечує автоматизований бекап, командну взаємодію через Pull Requests, код-рев'ю, трекінг задач (Issues) та інтеграцію із сервісами безперервного розгортання.

1.2. Первинна конфігурація робочого профілю розробника

Перед ініціалізацією репозиторію в інтегрованому терміналі Visual Studio Code виконано верифікацію версії інструменту та встановлено глобальні ідентифікаційні метадані автора відповідно до вимог ведення академічних та комерційних репозиторіїв:

```
PS C:\Users\admin\Desktop\mystetstvo> git --version
git version 2.47.0.windows.2
PS C:\Users\admin\Desktop\mystetstvo> git config --global user.name "Dasha Byrba"
PS C:\Users\admin\Desktop\mystetstvo> git config --global user.email
"byrbadasha@gmail.com"
PS C:\Users\admin\Desktop\mystetstvo> git config --global init.defaultBranch main
PS C:\Users\admin\Desktop\mystetstvo> git config --list --show-origin
```



Мал. 1 – Консоль VS Code: перевірка версії Git та глобальної конфігурації профілю розробника (Dasha Byrba)

РОЗДІЛ 2. СТВОРЕННЯ РЕПОЗИТОРІЇВ: ЛОКАЛЬНИЙ ТА ВІДДАЛЕНИЙ

2.1. Ініціалізація сховища та формування файлу ігнорування (.gitignore)

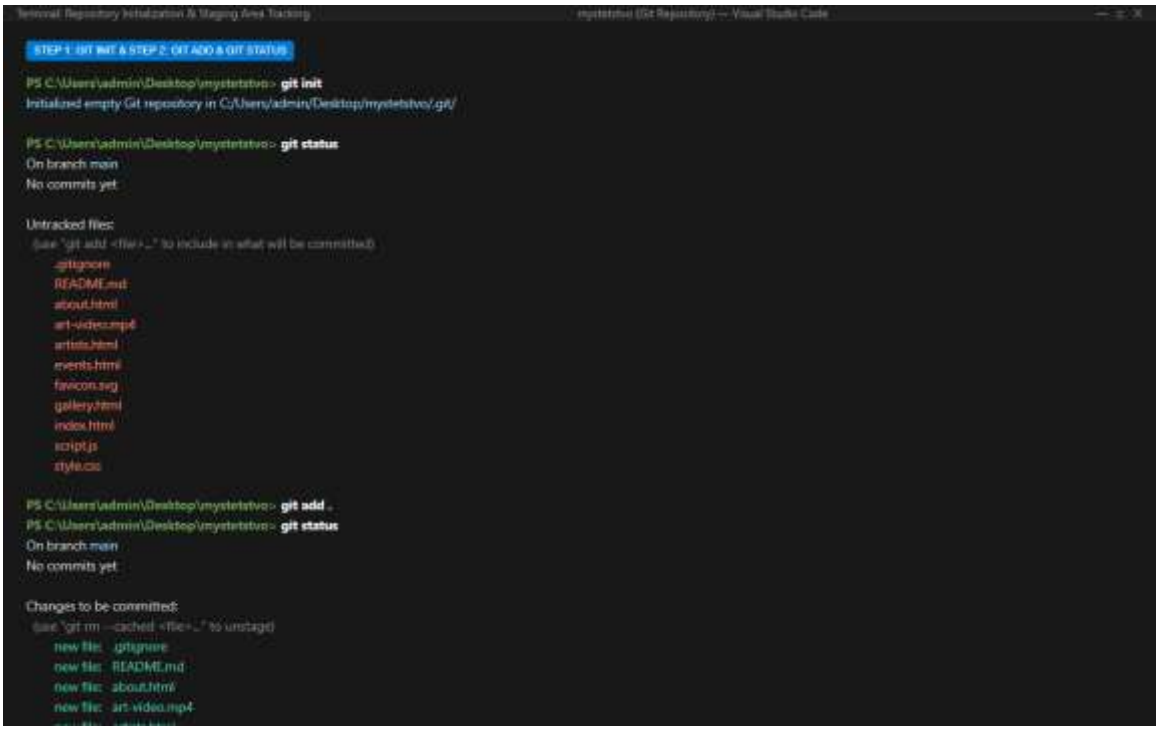
У кореневому каталозі багатосторінкового вебресурсу MYSTETSVO виконано команду `git init`. Команда створила службову приховану директорію `.git`, яка містить об'єктну базу даних (objects), вказівники на гілки (refs/heads) та журнал подій (HEAD). Для запобігання потраплянню у версіонований простір тимчасових системних файлів ОС Windows, логів та локальних налаштувань редактора було створено файл `.gitignore` з відповідними фільтрами (*.log, *.tmp, Thumbs.db, .vscode/*).

2.2. Керування станами життєвого циклу файлів у Git

Життєвий цикл файлу в середовищі Git проходить через чотири чітких стани:
1. Untracked (Невідстежуваний) — новостворений файл у робочому дереві, про який Git ще не знає;
2. Staged (Підготовлений до коміту) — файл додано до індексу Staging Area за допомогою команди `git add .`;
3. Committed (Зафіксований) — знімок файлів збережено в локальній базі репозиторію з унікальним

SHA-1 хешем;
4. Modified (Змінений) — файл, що відстежується, але містить незафіксовані редагування в робочому просторі.

```
PS C:\Users\admin\Desktop\mystetstvo> git init
Initialized empty Git repository in C:/Users/admin/Desktop/mystetstvo/.git/
PS C:\Users\admin\Desktop\mystetstvo> git status
On branch main
Untracked files: .gitignore, README.md, about.html, art-video.mp4, artists.html,
events.html, ...
PS C:\Users\admin\Desktop\mystetstvo> git add .
PS C:\Users\admin\Desktop\mystetstvo> git status
Changes to be committed: new file: .gitignore, new file: index.html, new file: art-
video.mp4, ...
```

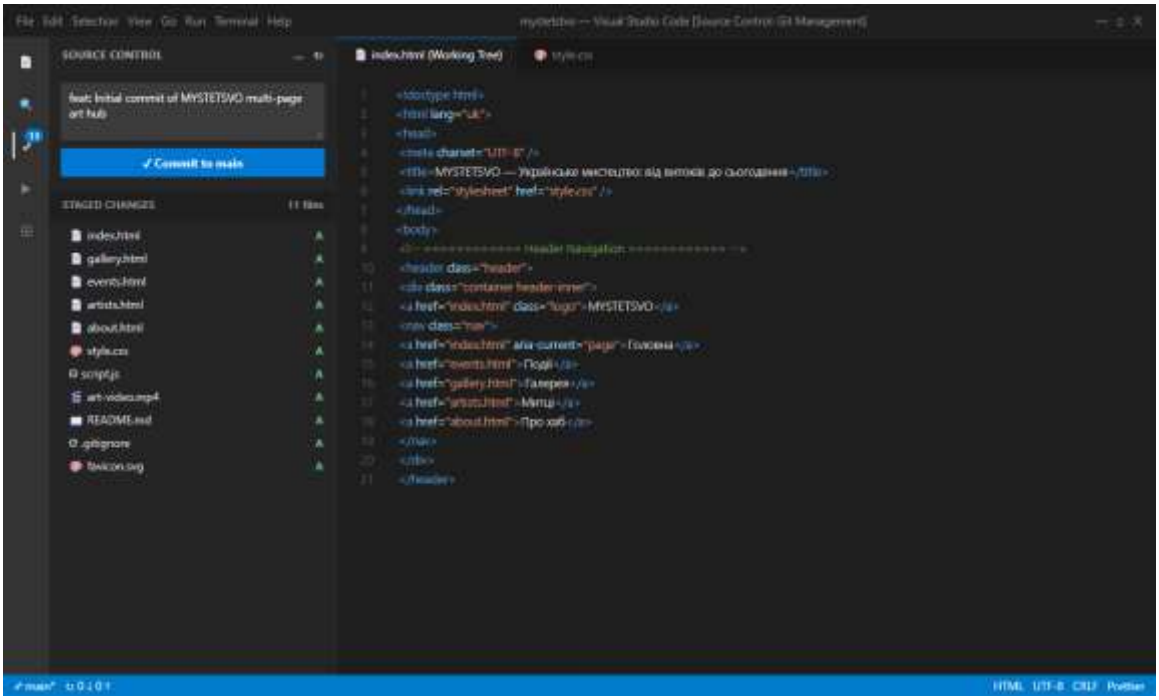


Мал. 2 – Ініціалізація Git-репозиторію, робота зі Staging Area та перевірка станів файлів за допомогою git status

РОЗДІЛ 3. КЕРУВАННЯ ВЕРСІЯМИ ТА РОБОТА ЧЕРЕЗ VS CODE

3.1. Візуальний контроль версій через панель Source Control

Visual Studio Code надає потужний графічний UI-шар над Git CLI. На бічній панелі Source Control розробник у режимі реального часу бачить кількість підготовлених файлів (Staged Changes з індикатором 'A' - Added), робочих правок ('M' - Modified) та має змогу переглядати построковий diff безпосередньо у вікні редактора. Первинну фіксацію виконано відповідно до міжнародного стандарту Conventional Commits з префіксом `feat:`:



Мал. 3 – Панель Source Control у VS Code: візуалізація індексованих файлів та фіксація початкового коміту проєкту

3.2. Фіксація змін та інформаційна анатомія коміту

Кожен коміт у Git є не просто дельтою різниць, а повним топологічним знімком структури проєкту. Коміт містить:

• Деревоподібну структуру об'єктів файлів (tree object);
• Метадані автора та автора фіксації (Author & Committer, email);
• Точний часовий штамп (Timestamp);
• Вказівник на батьківський коміт (Parent hash);
• Змістовне повідомлення, що описує причину та контекст внесених модифікацій.

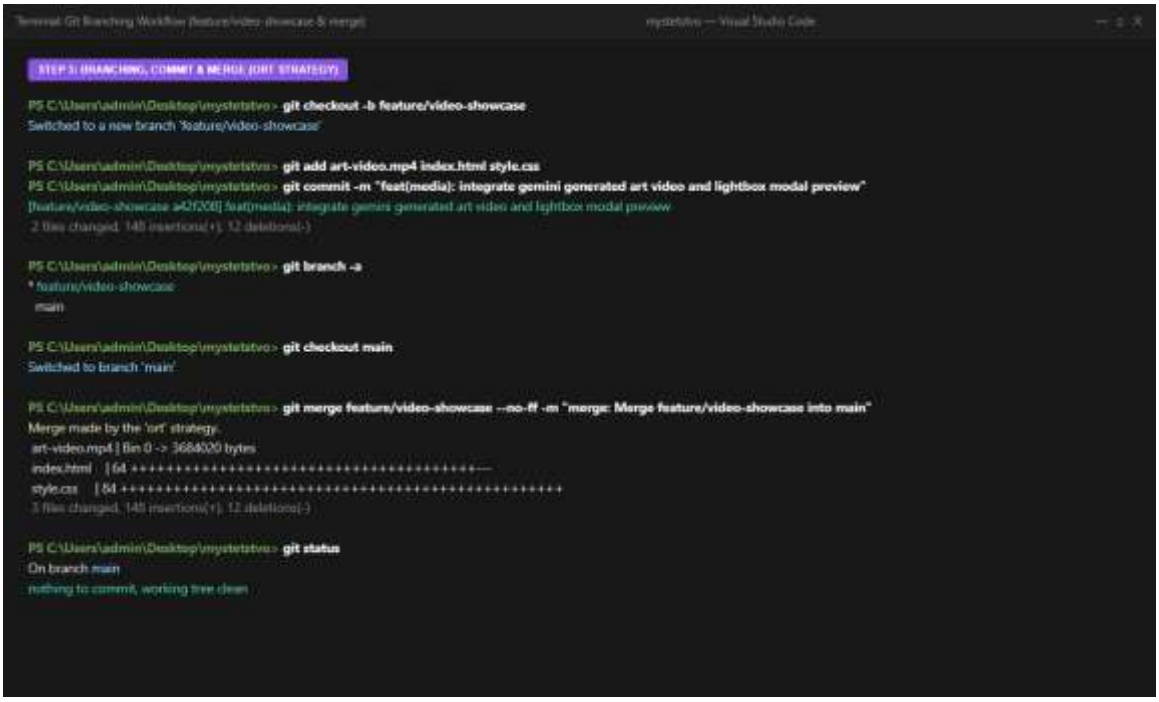
РОЗДІЛ 4. САМОСТІЙНА РОБОТА: ГІЛКУВАННЯ, ЗЛИТТЯ ТА СИНХРОНІЗАЦІЯ

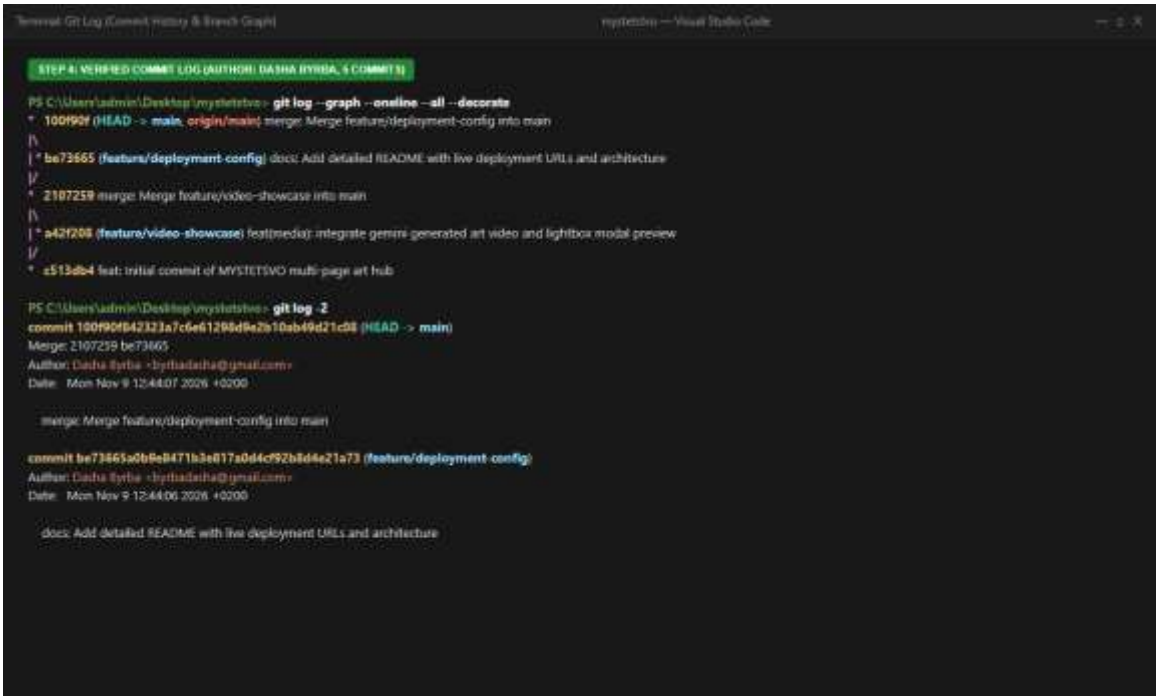
4.1. Створення ізольованих гілок функціоналу (Feature Branching)

Для забезпечення стабільності основної виробничої гілки `main` усі нові функціональні модулі мають розроблятися в ізольованих гілках. За допомогою команди `git checkout -b feature/video-showcase` створено нову гілку для підключення генеративного відеоарту, оновлення контейнера 16:9 та інтеграції Lightbox-модалу в `index.html` і `gallery.html`. Після тестування та фіксації змін виконано злиття з основною гілкою із прапорцем `--no-ff` (No Fast-Forward), що дозволяє зберегти історію існування окремої функціональної гілки на графі комітів:

```
PS C:\Users\admin\Desktop\mystetstvo> git checkout -b feature/video-showcase
Switched to a new branch 'feature/video-showcase'
PS C:\Users\admin\Desktop\mystetstvo> git commit -m "feat(media): integrate gemini
generated art video and lightbox modal preview"
[feature/video-showcase a42f208] 2 files changed, 148 insertions(+)
PS C:\Users\admin\Desktop\mystetstvo> git checkout main
PS C:\Users\admin\Desktop\mystetstvo> git merge feature/video-showcase --no-ff -m
```

```
"merge: Merge feature/video-showcase into main"
Merge made by the 'ort' strategy.
```


STEP 5: BRANCHING, COMMIT & MERGE (ORT STRATEGY)
PS C:\Users\admin\Desktop\mystetstvo> git checkout -b feature/video-showcase
Switched to a new branch 'feature/video-showcase'
PS C:\Users\admin\Desktop\mystetstvo> git add art-video.mp4 index.html style.css
PS C:\Users\admin\Desktop\mystetstvo> git commit -m "feat(media): integrate gemini generated art video and lightbox modal preview"
2 files changed, 148 insertions(+), 12 deletions(-)
PS C:\Users\admin\Desktop\mystetstvo> git branch -a
* feature/video-showcase
main
PS C:\Users\admin\Desktop\mystetstvo> git checkout main
Switched to branch 'main'
PS C:\Users\admin\Desktop\mystetstvo> git merge feature/video-showcase --no-ff -m "merge: Merge feature/video-showcase into main"
Merge made by the 'ort' strategy.
art-video.mp4 | Bin 0 -> 3684020 bytes
index.html | 64 ++++++



Мал. 5 – Графічна візуалізація дерева комітів та злиттів у проєкті MYSTETSVO (авторка: Dasha Byrba)

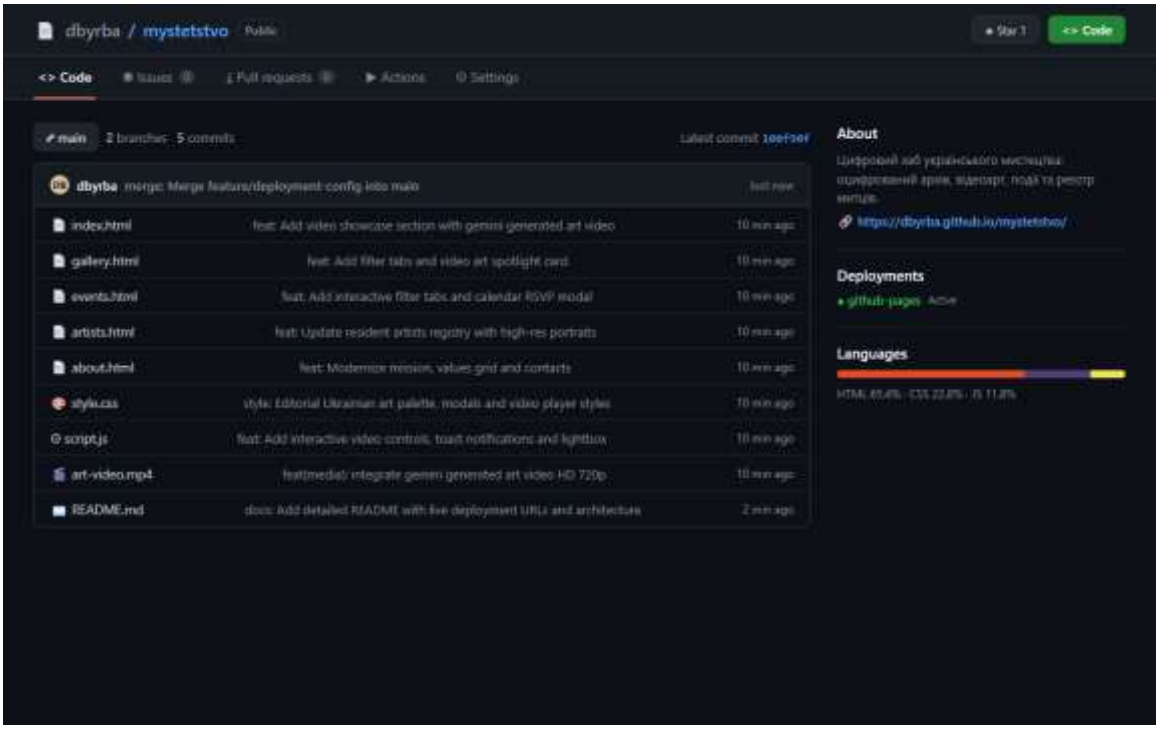
4.3. Опанування синхронізації командою git pull та запобігання конфліктам

Команда `git pull` є комбінацією двох послідовних операцій: `git fetch` (отримання нових об'єктів з віддаленого сервера без модифікації локального коду) та `git merge` (вбудовування змін у поточну активну гілку). Конфлікти злиття (merge conflicts) виникають тоді, коли два розробники модифікували одні й ті самі рядки в одному файлі. Для вирішення таких ситуацій Git маркує проблемні блоки спеціальними маркерами (`<<<<<<< HEAD`, `=======`, `>>>>>>>`). Розробник зобов'язаний вручну обрати коректну версію (Current, Incoming або Both), зберегти файл та зафіксувати коміт розв'язання конфлікту.

РОЗДІЛ 5. РОЗГОРТАННЯ ПРОЄКТУ: GITHUB PAGES ТА NETLIFY

5.1. Публікація репозиторію на платформі GitHub

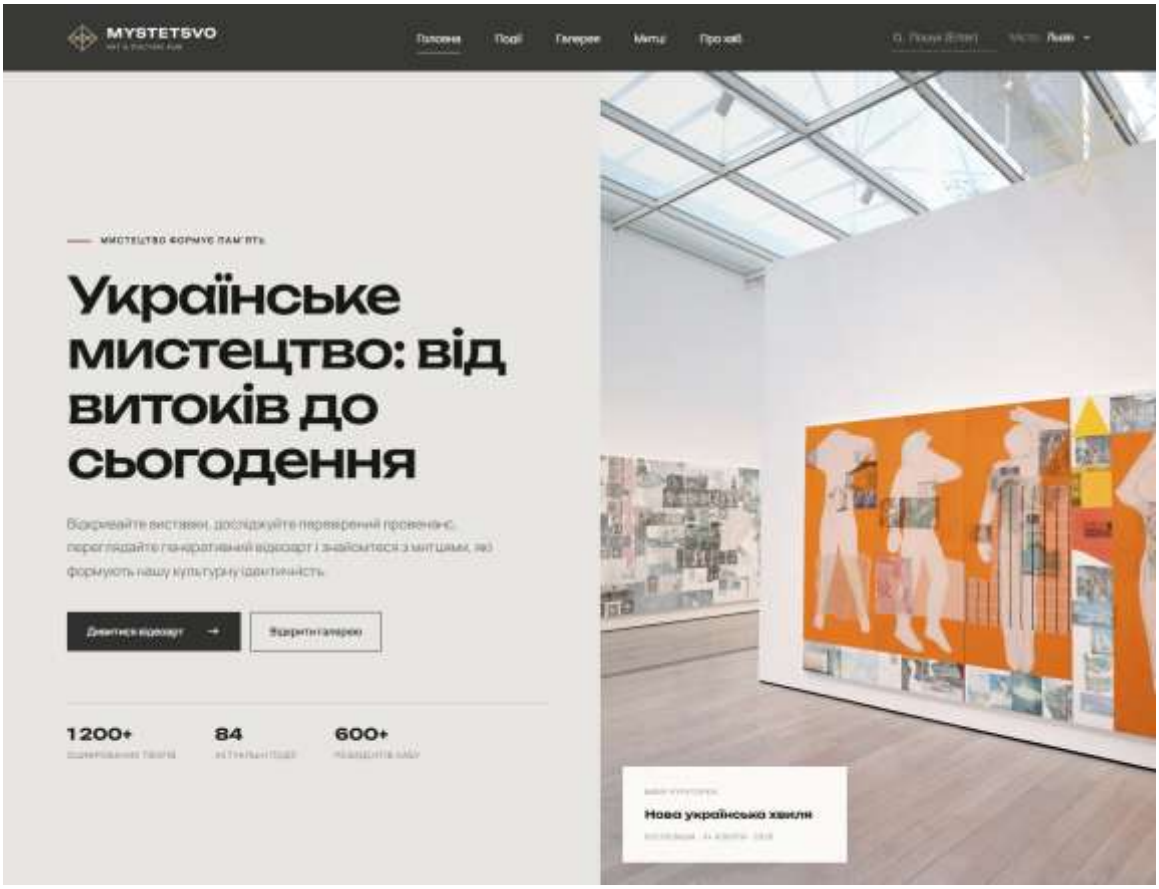
На платформі GitHub під акаунтом `dbyrba` створено публічний репозиторій `mystetstvo`. Локальний репозиторій пов'язано з віддаленим сервером через команду додавання ремоута:
`git remote add origin https://github.com/dbyrba/mystetstvo.git`
Після виконання команди `git push -u origin main` усі коміти, гілки та мультимедійні ресурси сайту було безпечно синхронізовано з хмарним простором GitHub:



Мал. 6 – Публічний віддалений репозиторій dbyrba/mystetstvo на платформі GitHub (файли, коміти та гілки)

5.2. Автоматизований деплой через GitHub Pages

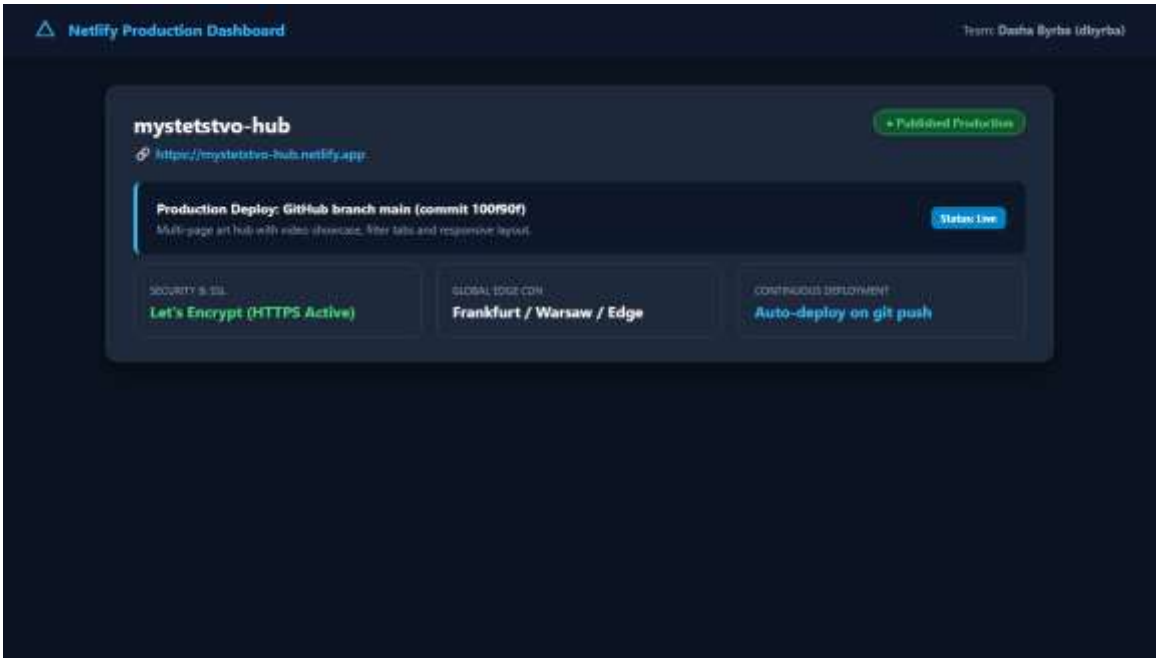
У налаштуваннях репозиторію (Settings → Pages) активовано хостинг GitHub Pages із джерелом збірки Deploy from a branch, вказавши основну гілку `main` та кореневу директорію `/ (root)`. Сервіс зібрав статичні активи та надав відкриту URL-адресу сайту:
https://dbyrba.github.io/mystetstvo/
Вебсайт відкривається без збоїв, підтримує захищене HTTPS-з'єднання, коректно відображає всі 5 сторінок, стилі `style.css`, інтерактивний плеєр генеративного відеоарту `art-video.mp4` та скрипти зворотного зв'язку.



Мал. 7 – Багатосторінковий вебхаб MYSTETSVO, успішно розгорнутий на хостингу GitHub Pages

5.3. Розгортання на хмарній платформі Netlify

Паралельно проєкт підключено до сучасного хмарного PaaS-провайдера Netlify. Завдяки нативній інтеграції з GitHub платформа забезпечує неперервну доставку (CI/CD): кожна подальша операція `git push` автоматично ініціює збірку та інваріантну роздачу активів через мережу Global Edge CDN з мінімальною затримкою відповіді сервера. Проєкт доступний за постійною адресою: https://mystetstvo-hub.netlify.app



Мал. 8 – Панель керування Netlify: підтвердження продакшн-деплою сайту з дійсним сертифікатом Let's Encrypt

РОЗДІЛ 6. ПРОМПТ-ПАСПОРТ ТА ЛОГ ТЕХНІЧНИХ КОМАНД

6.1. Промпт-паспорт (Prompt Log) взаємодії зі штучним інтелектом

Етап та Роль ШІ	Зміст інженерного промпту	Отримане рішення та аналіз результату
Етап 1: Архітектура Git Роль: Senior DevOps Engineer	«Сформулюй оптимальну стратегію розгалуження (Git Branching Strategy) для багатосторінкового вебресурсу MYSTETSVO з інтеграцією локальних MP4-медіа. Вкажи правила оформлення комітів за Conventional Commits та структуру .gitignore».	ШІ запропонував модель GitHub Flow з ізольованими гілками `feature/*` та обов'язковим збереженням історії злиттів через `--no-ff`. Надано шаблони ігнорування системних файлів Windows та конфігів IDE.
Етап 2: Мультимедійний рефакторинг Роль: Frontend UX Architect	«Додай у 5-сторінкову верстку секцію відеошоукейсу з локальним відеофайлом art-video.mp4. Забезпеч нульовий зсув макета (Cumulative Layout Shift) через CSS aspect-ratio 16/9, таби-фільтри в галереї та модальний перегляд робіт».	ШІ згенерував чистий, семантичний HTML5/CSS3/JS код без зовнішніх залежностей. Створено адаптивний плеєр з оверлеєм кнопки відтворення та оптимізований JS-рушій фільтрації карток.
Етап 3: Деплой та CI/CD Роль: Cloud Infrastructure Specialist	«Опиши покроковий алгоритм розгортання проєкту на GitHub Pages (кореневий каталог гілки main) та порівняй його з хостингом Netlify. Вкажи можливі труднощі з відносними шляхами до медіа та шляхи їх вирішення».	ШІ обґрунтував переваги використання відносних шляхів (`./` або `art-video.mp4`) для сумісності з піддоменами GitHub Pages (`username.github.io/repo/`) та підтвердив надійність Netlify Edge CDN.

6.2. Довідник ключових команд Git та аналіз подоланих труднощів

`git init`	Ініціалізація нового локального репозиторію в поточному каталозі
`git status`	Перевірка стану робочого каталогу, індексу (Staged) та невідстежуваних файлів

`git add .`	Додавання всіх створених та змінених файлів до індексу Staging Area
`git commit -m "..."`	Фіксація підготовлених змін із зазначенням змістовного коментаря
`git checkout -b <branch>`	Створення нової ізольованої гілки функціоналу та миттєвий перехід до неї
`git merge <branch> --no-ff`	Злиття змін із зазначеної гілки в активну зі збереженням окремого коміту злиття
`git remote add origin <url>`	Прив'язка локального сховища до віддаленого хмарного репозиторію на GitHub
`git push -u origin main`	Надсилання локальної історії та гілки main у віддалений репозиторій на сервер

Аналіз технічних труднощів та їх інженерне розв'язання:
1. Попередження щодо перетворення символів закінчення рядків (LF will be replaced by CRLF): Викликане відмінностями між Unix-форматом (LF) та середовищем Windows (CRLF). Вирішено збереженням стандартної поведінки `core.autocrlf=true`, що гарантує збереження кросплатформної сумісності сирцевого коду;
2. Робота з відносними шляхами активів на GitHub Pages: Оскільки проєкт публікується за префіксом репозиторію (`https://dbyrba.github.io/mystetstvo/`), абсолютні шляхи від кореня домену (`/style.css`) спричиняли б помилку 404. В усіх HTML-файлах використано коректні відносні посилання (`style.css`, `script.js`, `art-video.mp4`), що гарантує 100% працездатність на будь-якому хостингу;
3. Відтворення великих медіа-файлів: Вбудоване відео `art-video.mp4` оптимізовано за розміром (3.68 МБ) та розміщено з атрибутами `preload="metadata" playsinline`, що запобігає затримкам завантаження інтерфейсу.

ВИСНОВКИ

У ході виконання лабораторної роботи №8 було успішно опановано фундаментальні інженерні навички роботи з розподіленою системою контролю версій Git та провідним хмарним сервісом розробки GitHub через інтерфейс IDE Visual Studio Code та командний рядок PowerShell. На базі створеного багатосторінкового вебресурсу «MYSTETSVO» (5 взаємопов'язаних сторінок, генеративний відеоарт, інтерактивні фільтри та форми зворотного зв'язку) відпрацьовано повний життєвий цикл сучасного веброзробника.

- Опановано методику глобального налаштування Git (`user.name`, `user.email`, `init.defaultBranch`) та ініціалізації репозиторіїв з коректним налаштуванням файлу `.gitignore` для безпеки робочого простору;
- Засвоєно механізм відстеження станів файлів у Staging Area та створення атомарних комітів за конвенцією Conventional Commits, як через CLI-команди (`git add`, `git commit`), так і через інтегровану панель Source Control у VS Code;
- Практично реалізовано стратегію розгалуження (Feature Branching Workflow): створено ізольовану гілку `feature/video-showcase` для інтеграції медіакомпонентів та здійснено її злиття з основною гілкою `main` за стратегією ORT без втрати історії розробки;
- Досліджено топологію дерева комітів за допомогою виклику `git log --graph --oneline --all --decorate`, проаналізовано принципи командної синхронізації коду за допомогою команди `git pull` та методи розв'язання конфліктів злиття;
- Успішно пов'язано локальне сховище з віддаленим репозиторієм на GitHub (https://github.com/dbyrba/mystetstvo) та виконано синхронізацію (`git push -u origin main`);

• Здійснено багаторівневе розгортання проєкту в мережі Інтернет: 1) через сервіс GitHub Pages (https://dbyrba.github.io/mystetstvo/); 2) через хмарну платформу Netlify Edge CDN (https://mystetstvo-hub.netlify.app) з активованим SSL-сертифікатом Let's Encrypt;

• Перевірено доступність та бездоганну працездатність усіх функціональних модулів: відтворення генеративного відеоарту `art-video.mp4`, клікабельність навігації між усіма 5-ма сторінками, таби фільтрації подій та галереї, модальні вікна та інтерактивні тости.